

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

BCSE497J — PROJECT I

PROJECT REPORT

on

MEMORYSCAPE

A Spatiotemporal Personal Knowledge Graph with Biologically Inspired Recall

Prepared by

Akil — 25BCE2777

Syon Vijae Thyvalappil — 25BCE2598

School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

October 2026

Abstract

MemoryScape is a local-first personal knowledge graph that stores episodic memories with event time, geographic position, contextual tags, and explicit directed associations. It makes three database-native recall operations inspectable: PostGIS route-corridor retrieval ordered along a user-drawn path; exponential strength decay with transactional reinforcement; and bounded, cycle-safe spreading activation implemented in recursive SQL. A PostgreSQL materialized view supports review of fading memories, with pg_cron scheduling decay and refresh jobs. A TypeScript/Next.js interface exposes the algorithms through a map, timeline, graph, memory library, and review workspace. The prototype uses deterministic synthetic data and does not require machine learning, vector search, or paid services. Validation exercised PostgreSQL 16.15, PostGIS 3.6.4, 1,200 seeded records, database algorithm tests, concurrency checks, application tests, and browser workflows. Additional five-trial synthetic measurements extend to 50,000 memories. The results demonstrate functional feasibility and workload-dependent query behavior; they do not establish cognitive or biological validity, user benefit, or a general spatial-index speedup. A cross-browser run passed on Chromium and Firefox; the WebKit run had four environment/UI-test failures and is reported as incomplete.

Keywords: PostgreSQL; PostGIS; personal knowledge graph; spatiotemporal retrieval; recursive SQL; temporal decay; materialized view.

Table of Contents

Abstract 2
1. Introduction 4
2. Literature Study 4
3. Objectives and Research Questions 5
4. System Design 5
5. Implementation and Evaluation 6
6. Conclusion and Future Work 8
References 8

1. Introduction

1.1 Problem and motivation

Personal notes commonly keep text while losing the context that helps a person find it again: where an episode happened, when it occurred, which people or topics were involved, and which other episodes it evokes. MemoryScape models these elements explicitly in a relational database so that recall can be queried, explained, measured, and reproduced.

1.2 Scope and design stance

The system is a research prototype for one trusted local user. Its biological language describes engineering analogies, not validated models of human cognition. No semantic embedding, machine-learning model, LLM, vector database, or remote inference API is part of retrieval. OpenStreetMap-compatible tiles provide visual context only; the memory records and retrieval computation remain local.

1.3 Contributions

- An integrated PostgreSQL design joining normalized episode, context, association, reinforcement, and spatial data.
- A PostGIS route walk with meter-based proximity, temporal interval filtering, and SQL route ordering.
- A transactional temporal model that applies elapsed decay before reinforcement and records each reinforcement event.
- A recursive SQL spreading-activation query with bounded depth, threshold pruning, cycle prevention, and winning-path explanations.
- A scheduled fading-memory review view plus a reproducible synthetic evaluation harness.

2. Literature Study

2.1 Forgetting and reinforcement as computational analogies

Ebbinghaus’s experimental work established a quantitative tradition for studying retention over time. MemoryScape uses a simple exponential function as an explicit, inspectable engineering model. The chosen decay rate is per memory and configurable; it is not fitted to participants or claimed to reproduce a biological forgetting curve.

2.2 Associative activation

Collins and Loftus described semantic memory retrieval through spreading activation over associations. MemoryScape borrows the graph-traversal analogy while retaining explicit edge weights, signed valence, path traces, and deterministic pruning in SQL. The prototype’s coefficients are implementation parameters rather than cognitive constants.

2.3 Spatial and temporal context

Cognitive-map research motivates treating spatial relationships as useful cues. In this implementation, PostGIS geography and a route geometry provide a concrete query substrate: distance is measured in meters, proximity uses ST_DWithin, and ST_LineLocatePoint gives an along-route ordering key.

2.4 Database contribution and gap

PostgreSQL recursive common table expressions, transaction isolation, GiST indexes, stored routines, materialized views, and pg_cron provide the computation without a separate graph service. The contribution is the assembled, explainable database-native prototype and its evaluation workflow; this project does not claim a new cognitive theory or clinical outcome.

3. Objectives and Research Questions

- RQ1: Can PostGIS retrieve memories near user-drawn routes and order them deterministically along the path?
- RQ2: Can decay and reinforcement be applied atomically and idempotently in PostgreSQL?
- RQ3: Can recursive SQL provide bounded, explainable associative retrieval with cycles and weak paths handled in-database?
- RQ4: How do scan strategy, route selectivity, traversal depth, and pruning affect measured work on deterministic datasets?
- RQ5: Can a materialized view lower the cost of listing fading memories compared with recomputing review priority?

The project tests software correctness and synthetic-workload behavior. Human recall effectiveness, usability, and biological validity remain open empirical questions and require a separately approved study.

4. System Design

4.1 Architecture

The browser sends validated inputs to a TypeScript API. Parameterized SQL and database functions perform the substantive computation. The frontend renders returned records and explanations; it does not implement graph traversal, geographic filtering, or temporal strength updates.

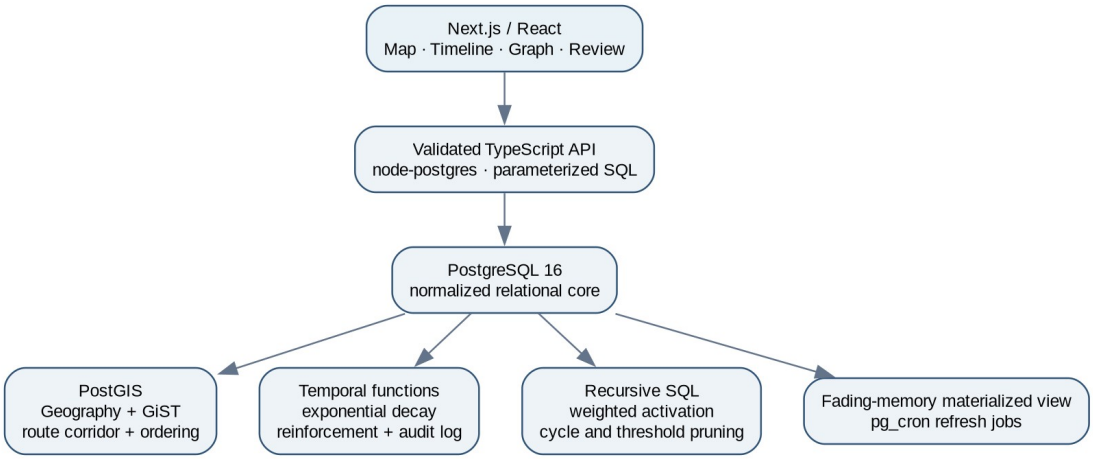


Figure 1. Implemented database-centered system architecture.

4.2 Relational model and integrity

The normalized core separates memories from reusable contexts and represents their many-to-many association in memory_contexts. Directed edges are stored in memory_edges; reinforcement changes are auditable in memory_reinforcements. UUID primary keys, foreign keys, uniqueness constraints, range checks, timestamp defaults, and purpose-specific indexes enforce core invariants. A geography(Point,4326) column and GiST index support spatial search.

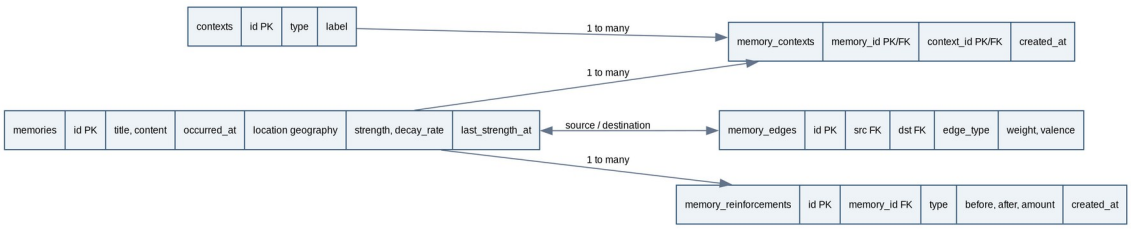


Figure 2. Core entity relationships; edge endpoints both reference memories.

4.3 Spatial walk

A validated LineString is converted to geography. The SQL routine filters with ST_DWithin(memory.location, route, radius_m), computes ST_Distance in meters, and sorts by ST_LineLocatePoint. The spatiotemporal extension additionally applies a half-open event interval: occurred_at ≥ from and occurred_at < until. UUID and timestamp tie-breaks make ordering stable.

4.4 Temporal decay and reinforcement

For elapsed time expressed in days, the implementation applies $S(t) = \text{clamp}(S_0 \times \exp(-\lambda\Delta t), 0, 1)$. The decay routine updates eligible rows only when the supplied logical time is later than last_strength_at; the strength and timestamp advance atomically. Reinforcement locks the memory row, first applies unapplied decay, then computes $S' = \min(1, S + \alpha(1 - S))$, and records before/after values in the same transaction. Linking an already-associated context does not trigger another reinforcement.

4.5 Associative recall and fading review

Recursive SQL propagates A_next = A_current × edge_weight × (0.75 + 0.25 × |valence|) × target_strength. The query prevents per-path revisits using a UUID path, prunes below threshold inside recursion, limits depth to three by default, and selects a deterministic strongest path for each destination. The fading_memories_mv materializes low-strength review candidates; pg_cron schedules daily decay and hourly view refresh.

5. Implementation and Evaluation

5.1 Technology and reproducibility

Layer	Implementation
Database	PostgreSQL 16.15; PostGIS 3.6.4; pg_cron; visible SQL migrations and stored functions
API	Node.js, TypeScript, node-postgres, runtime validation, parameterized SQL
Interface	Next.js 16, React 19, Tailwind CSS, Leaflet, Cytoscape.js
Quality/evaluation	Vitest, PostgreSQL integration tests, Playwright, deterministic seed and EXPLAIN (ANALYZE, BUFFERS) runners
Synthetic data	1,200 memory validation seed; benchmark artifacts at 10,000 and 50,000 records

5.2 Verification performed

Check	Observed result
Vitest application/unit suite	72 tests passed across 18 files
TypeScript / production build	tsc --noEmit passed; Next.js production build completed
Database initialization	Migrations 001–013; PostGIS/pg_cron routines verified; deterministic 1,200-memory seed
SQL correctness and research database tests	Research SQL contract passed; algorithm SQL assertions passed; rollback preserved fixture state
Concurrency and service checks	8-session reinforcement concurrency passed; push-delivery checks passed
Study/evaluation tooling	Study validation and DB verification passed; synthetic research evaluation completed
Playwright Chromium + Firefox	75/75 passed in each browser (150 total)
Playwright WebKit	72/75 passed; 3 Daily-page checks still fail in the WebKit browser runner
Authenticated browser check	1/1 passed with an ephemeral test password

After storage cleanup, docker compose up --build -d web started successfully against the existing PostgreSQL volume. The health endpoint reported PostgreSQL 16.15 and PostGIS 3.6.4; both pg_cron jobs were registered, and the existing 1,200 memory rows remained unchanged. No demo reseed was run.

5.3 Synthetic correctness and research measurements

The deterministic eight-memory ground-truth fixture produced an associative baseline F1 of 0.909 and recall of 1.000 across the labeled target set. These values describe the constructed fixture, not people. The application's SQL tests additionally cover boundaries, cycles, negative valence, depth caps, and transaction behavior.

5.4 Five-restart, 50,000-memory query measurements

Each observation is a one-repetition query after a PostgreSQL restart; five sequential trials were summarized by medians. Restarting resets PostgreSQL shared_buffers but does not clear the operating-system cache. The workload uses a synthetic dense campus route and should not be generalized to other distributions.

Query	Median execution (ms)	Shared reads median
Spatial index candidate, 150 m	58.784	354
Controlled sequential plan	49.212	213
Associative recall, depth 1	0.190	6
Associative recall, depth 2	0.166	4
Associative recall, depth 3	0.198	5
Fading materialized-view query	7.493	960
Equivalent direct fading query	205.908	0
Timeline query	0.091	0

At this workload, the selected bitmap index candidate was slower than the controlled sequential plan; this is evidence against claiming a general spatial-index speedup for the tested broad corridor. The materialized-view lookup was faster than recomputing the direct review query in these trials, while view creation and refresh had separate maintenance cost.

5.5 Spatial selectivity sweep

Radius	Returned rows (50K)	Share	Planner candidate median	Candidate plan
25 m	2,025	4.05%	129.883 ms	Bitmap index + heap scan
150 m	14,375	28.75%	134.925 ms	Bitmap index + heap scan
1,000 m	49,725	99.45%	159.016 ms	Parallel sequential scan

The five-trial confidence intervals for planner-candidate and controlled sequential means overlap at each radius. Scan selection changed as route selectivity rose, but this experiment does not establish a reliable causal benefit from the index. It uses one route and one synthetic cluster; operating-system cache was uncontrolled.

5.6 Threats to validity and limitations

- Synthetic records and labels do not represent personal diaries or user recall.
- No human-participant study has been conducted; cognitive validity, usefulness, accessibility outcomes, and privacy perceptions are not measured.
- Five sequential restart trials on one host have limited statistical power and no independent machine replication.
- The spatial workload is clustered and broad; selectivity and scan choices can differ on real data.
- The WebKit suite is incomplete (72/75); three Daily-page browser checks time out. Chromium and Firefox passed.

- The local Compose restart was blocked by host storage exhaustion, so a disposable PostgreSQL instance was used for integration validation.

6. Conclusion and Future Work

MemoryScape demonstrates a reproducible, database-centered implementation that combines spatial route recall, temporal decay and reinforcement, associative graph traversal, and fading-memory review. Its main strength is that the query logic is explicit in PostgreSQL and the interface can expose result paths, distances, timestamps, and strength changes. Current evidence supports functional correctness on the tested fixtures and measured query behavior on synthetic data; it does not establish biological plausibility, real-world usefulness, or a general performance advantage.

Next steps are to restore the local storage environment, complete the WebKit failures, test with larger varied spatial distributions and more independent hosts, and conduct a usability/recall study only after institutional ethics review and informed-consent preparation. Any tuning of decay or activation parameters should be treated as an empirical research question, not inferred from synthetic calibration.

References

[1] Ebbinghaus, H. (1885/1913). Memory: A Contribution to Experimental Psychology. Translated by H. A. Ruger and C. E. Bussenius. Teachers College, Columbia University. (Historical work; used here as context for quantitative forgetting models.)

[2] Collins, A. M., & Loftus, E. F. (1975). A spreading-activation theory of semantic processing. Psychological Review, 82(6), 407–428. https://doi.org/10.1037/0033-295X.82.6.407

[3] O’Keefe, J., & Nadel, L. (1978). The Hippocampus as a Cognitive Map. Clarendon Press, Oxford.

[4] PostgreSQL Global Development Group. PostgreSQL 16 Documentation: WITH Queries (Common Table Expressions). https://www.postgresql.org/docs/16/queries-with.html

[5] PostGIS Project Steering Committee. PostGIS Documentation: ST_DWithin; ST_LineLocatePoint; Using PostGIS: Spatial Indexes. https://postgis.net/docs/manual-dev/en/ST_DWithin.html ; https://postgis.net/docs/manual-3.7/en/ST_LineLocatePoint.html ; https://postgis.net/docs/using_postgis_dbmanagement.html

[6] MemoryScape project artifacts. Database schema, algorithm notes, implementation status, evaluation methodology, raw benchmark plans and deterministic research results. Repository documentation, captured 2026.